take in one or more arrays of data, and sometimes generators, and append attributes to the data necessary to draw it in certain positions or sizes, either statically or dynamically. You'll see some of the simpler layouts in chapter 5, and then focus on the force-directed network layout and other network layouts in chapter 6.

4.2 Creating an axis

Scatterplots, which you worked with in chapters 1 and 2, are a simple and extremely effective charting method for displaying data. For most charts, the x position is a point in time and the y position is magnitude. For example, in chapter 2 you placed your tweets along the x-axis according to when the tweets were made and along the y-axis according to their impact factor. In contrast, a scatterplot places a single symbol on a chart with its xy position determined by quantitative data for that datapoint. For instance, you can place a tweet on the y-axis based on the number of favorites and on the x-axis based on the number of retweets. Scatterplots are common in scientific discourse and have grown increasingly common in journalism and public discourse for presenting data such as the cost compared to the quality of health care.

4.2.1 Plotting data

Scatterplots require multidimensional data. Each datapoint needs to have more than one piece of data connected with it, and for a scatterplot that data must be numerical. You need only an array of data with two different numerical values for a scatterplot to work. We'll use an array where every object represents a person for whom we know the number of friends they have and the amount of money they make. We can see if having more or less friends positively correlates to a high salary.

```
var scatterData = [{friends: 5, salary: 22000},
{friends: 3, salary: 18000}, {friends: 10, salary: 88000},
{friends: 0, salary: 180000}, {friends: 27, salary: 56000},
{friends: 8, salary: 74000}];
```

If you think these salary numbers are too high or too low, pretend they're in a foreign currency with an exchange rate that would make them more reasonable.

Representing this data graphically using circles is easy. You've done it several times:

```
d3.select("svg").selectAll("circle")
   .data(scatterData).enter()
   .append("circle").attr("r", 5).attr("cx", function(d,i) {
           return i * 10;
   }).attr("cy", function(d) {
           return d.friends;
   });
```



Figure 4.3 Circle positions indicate the number of friends and the array position of each datapoint.